

CarePlus Support Tickets & Logs

End-to-End AWS ETL & Analytics Pipeline

Event-Driven Data Engineering for Insurance Support Operations

FINAL PROJECT REPORT — DATA ENGINEERING DOCUMENTATION PACKAGE

Prepared by: Sahajahanur Rahman Laskar

AWS · Lambda · Glue · S3 · Redshift Serverless · Power BI

August 2026

connectingsrl@gmail.com | linkedin.com/in/sahajahanur-laskar | github.com/Sahajahanur

Table of Contents

Table of Contents	2
1. Project at a Glance	4
2. Business Problem & Objectives	4
2.1 Business Problem	4
2.2 Business Solution	4
2.3 Before vs After	4
3. Business Process	4
4. Technical Architecture	5
4.1 Support Logs Path (Lambda-only)	5
4.2 Support Tickets Path (Lambda + Glue)	6
5. Data Flow & End-to-End Lineage	6
6. S3 Data Lake Design	7
7. Support Logs ETL.....	7
7.1 Incremental, date-tracked ingestion.....	7
7.2 Regex-based log parsing.....	8
7.3 Rule-based cleaning	8
7.4 Event-driven Lambda trigger	8
8. Support Tickets ETL (Lambda + Glue)	8
9. Data Models & Schemas	9
9.1 Support Logs — Core Schema.....	9
9.2 Support Tickets — Core Schema.....	9
9.3 Warehouse Model — Current State	9
10. Data Quality Framework	10
10.1 Data Quality Flow	10
11. Incremental Loading to Redshift.....	10
11.1 Current approach to idempotency.....	11
12. Monitoring & Observability	11
13. Operational Runbook	11
13.1 Scenario 1 — Lambda ETL Failure	11
13.2 Scenario 2 — Glue Job Failure	12
13.3 Scenario 3 — Redshift Load Failure (actual incident)	12
14. Alerting Strategy	12
14.1 Current implementation.....	12
14.2 Production target	12
15. Business Glossary.....	12

16. Security.....	13
16.1 Implemented in this build	13
16.2 Production-grade improvements.....	13
17. Cost Management / FinOps.....	13
17.1 Daily Shutdown Checklist	13
17.2 Design trade-offs.....	13
17.3 Verified teardown after validation.....	13
18. Validation & Reconciliation	14
19. Testing & Test Cases	14
20. Troubleshooting Case Study: psycopg2 in Lambda	15
20.1 Incident.....	15
20.2 Root cause	15
20.3 Resolution.....	15
20.4 Result.....	15
21. Power BI Business Analytics	15
21.1 Support Logs Dashboard	15
21.2 Ticket Insights Dashboard.....	16
22. Business Use Cases Enabled	16
22.1 Operations	16
22.2 Technology / Engineering.....	16
22.3 Management	17
23. AWS Resource Inventory — During Project Execution	17
24. Architecture Decisions & Trade-offs	17
Why not EC2?	17
25. Challenges & Root Causes	17
26. Limitations & Production Roadmap	18
26.1 Current limitations	18
26.2 Production-Grade Target Architecture — V2	18
27. Production Readiness Scorecard	18
28. How I Explain This Project in 90 Seconds	19
29. Conclusion	19
30. Author & Contact	19

1. Project at a Glance

Component	Implementation
Domain	Insurance support operations (tickets + system logs)
Sources	MySQL, daily .log files, daily .csv ticket exports
Storage	Amazon S3 (raw and processed zones)
ETL – Logs	AWS Lambda + Python (regex parsing, rule-based cleaning)
ETL – Tickets	AWS Lambda trigger + AWS Glue PySpark (Visual ETL)
File format	Parquet + Snappy compression
Warehouse	Amazon Redshift Serverless
BI layer	Power BI
Trigger model	S3 Object Created events (fully event-driven)
Validation	AWS Glue Data Quality + manual row-count reconciliation
Monitoring	Amazon CloudWatch
Architecture style	Event-driven, serverless, incremental

2. Business Problem & Objectives

2.1 Business Problem

Insurance support operations continuously generate two related but structurally different data streams. Support tickets capture customer-facing issues — priority, agent, channel, interactions, and resolution outcome. Support logs capture system-level events — log level, response time, CPU utilization, and originating user agent.

Processed manually, this data creates real operational cost: delayed reporting, inconsistent cleansing, duplicate and invalid records, and slow root-cause identification when something goes wrong in production.

2.2 Business Solution

Automate ingestion, transformation, validation, and warehouse loading end-to-end, so that new operational data becomes analytics-ready within minutes of arriving — with no manual intervention — and is immediately queryable in Power BI for both operations and management audiences.

2.3 Before vs After

Area	Before (Manual)	After (This Pipeline)
Ingestion	Manual file handling	Event-driven (S3 trigger)
Transformation	Manual cleaning per batch	Automated Lambda / Glue ETL
Validation	Ad hoc, inconsistent	Automated Data Quality gate
Warehouse load	Full manual reload	Incremental, event-triggered
Reporting	Delayed	Refresh-ready in Power BI
Troubleshooting	Difficult to trace	CloudWatch end-to-end traceability

3. Business Process

Why the pipeline exists, in business terms rather than AWS terms: a customer interaction or a system event becomes raw data, is automatically turned into trustworthy analytics, and is used by two different audiences to improve support performance.

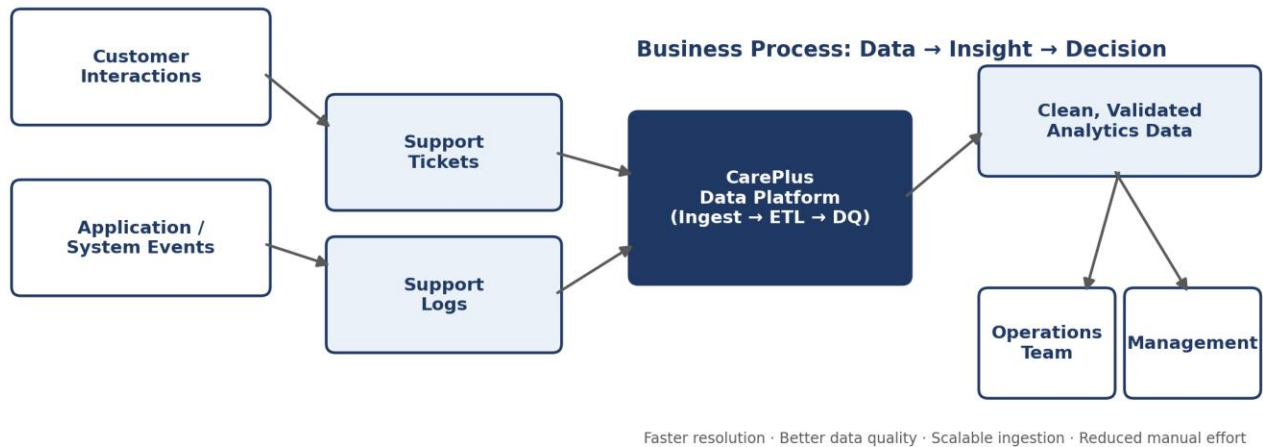


Figure 1: Business process — how raw operational events become decisions.

4. Technical Architecture

The pipeline runs two parallel processing paths — one for logs, one for tickets — that converge on a shared processed-data layer, then fan out to both an ad-hoc SQL layer (Athena) and a structured analytical warehouse (Redshift Serverless) feeding Power BI.

Pipeline Architecture

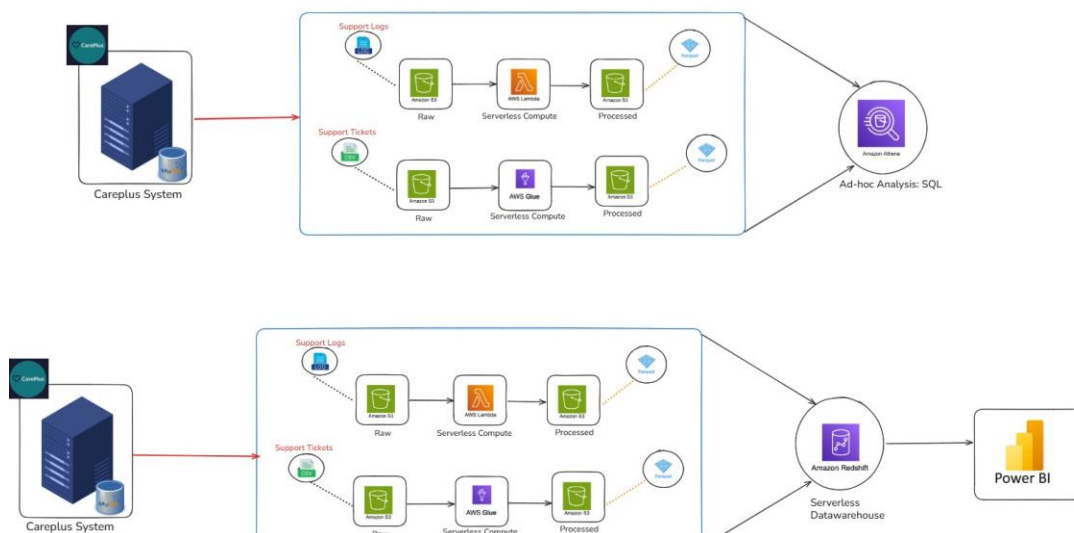


Figure 2: End-to-end AWS architecture — dual ingestion paths, Parquet processed zone, Athena ad-hoc layer, and Redshift → Power BI serving layer.

4.1 Support Logs Path (Lambda-only)

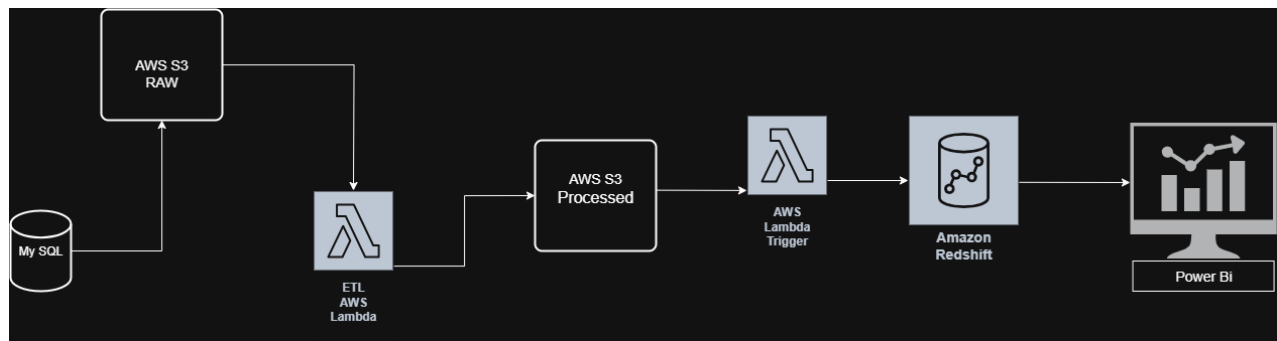


Figure 3: Support Logs pipeline — MySQL/log source → S3 Raw → Lambda ETL → S3 Processed → Lambda trigger → Redshift → Power BI.

4.2 Support Tickets Path (Lambda + Glue)

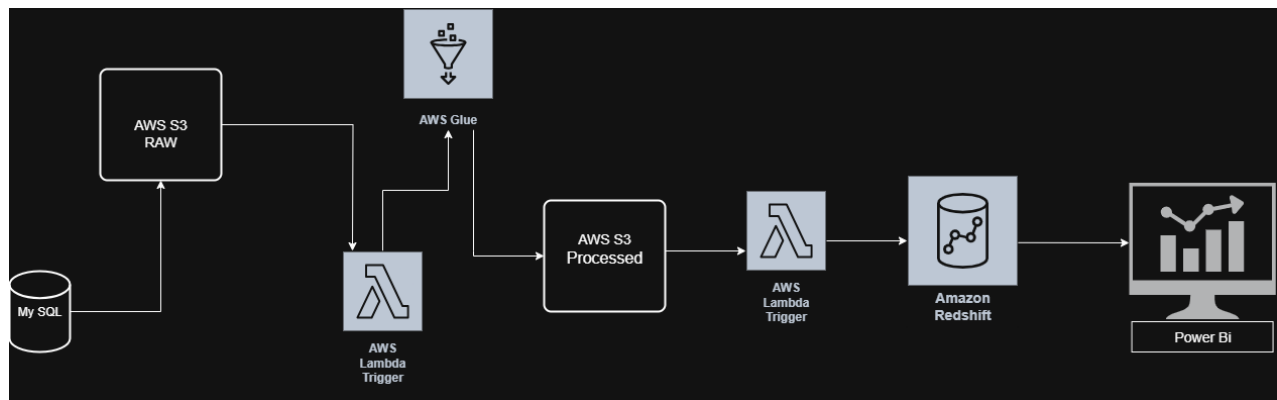


Figure 4: Support Tickets pipeline — adds an AWS Glue PySpark stage between S3 Raw and S3 Processed for heavier, structured-data transformation.

5. Data Flow & End-to-End Lineage

- Source data (MySQL export / daily .log / daily .csv) is uploaded to S3 Raw via a date-tracked Python ingestion notebook.
- An S3 Object Created event fires on arrival of a new .log or .csv file.
- Logs route to Lambda for direct regex-based ETL; tickets route to Lambda, which invokes an AWS Glue PySpark job for schema standardization.
- Cleaned output is written to S3 Processed as Parquet (Snappy-compressed) and passed through a Data Quality gate.
- A second, incremental Lambda is triggered on the processed-zone write and loads only the new records into Amazon Redshift Serverless.
- Power BI connects to Redshift and refreshes to reflect the newly loaded records, closing the loop from raw file to business dashboard.

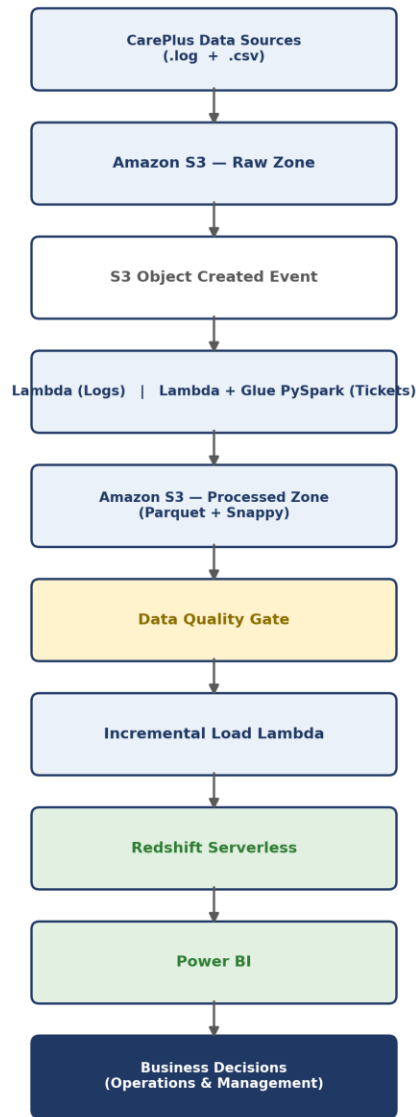


Figure 5: End-to-end data lineage, source to business decision.

6. S3 Data Lake Design

Zone	Path pattern	Contents
Raw – Logs	support-logs/raw/	Daily .log files, delimiter-separated free text
Raw – Tickets	support-tickets/raw/	Daily .csv ticket exports
Processed – Logs	support-logs/processed/	Cleaned, validated Parquet output
Processed – Tickets	support-tickets/processed/	Cleaned, schema-standardized Parquet output

Bucket: careplus-data-srl (ap-southeast-1). The current layout is source-first (support-logs/, support-tickets/) rather than zone-first (raw/, processed/); this is flagged in Section 25 as a planned restructure for cleaner partition discovery at scale.

7. Support Logs ETL

7.1 Incremental, date-tracked ingestion

Each ingestion run reads the last successfully uploaded date from a tracker file, computes the next date, uploads that file, and advances the tracker — preventing re-ingestion of already-processed dates and keeping batches deterministic.

```
last_date = read_last_date(DATE_TRACKER_FILE)
next_date = get_next_date(last_date)

file_name = f"support_logs_{next_date}.log"
s3_key = f"{S3_PREFIX}{file_name}"

upload_log_file_to_s3(file_name, S3_BUCKET, s3_key)
update_last_date(DATE_TRACKER_FILE, next_date)
```

7.2 Regex-based log parsing

Raw log files are split on a --- delimiter into individual entries, then a regex pattern extracts 14 structured fields from each free-text entry (see Section 8 for the documented subset used downstream).

```
entries = [e.strip() for e in raw_logs.split('---') if e.strip()]

for entry in entries:
    match = log_pattern.search(entry)
    if match:
        parsed_entries.append(match.groupdict())
```

7.3 Rule-based cleaning

```
df = df.drop_duplicates()

df['log_level'] = df['log_level'].replace({
    'INFØ': 'INFO', 'DEBG': 'DEBUG',
    'warnING': 'WARNING', 'EROR': 'ERROR'
})

df = df[df['response_time'] >= 0]
```

7.4 Event-driven Lambda trigger

The Lambda reads the bucket and object key directly from the S3 event payload rather than a hard-coded filename, so it works for any newly arrived file matching the trigger suffix.

```
record = event['Records'][0]
bucket = record['s3']['bucket']['name']
key = record['s3']['object']['key']

print(f"Triggered by s3://{bucket}/{key}")
```

8. Support Tickets ETL (Lambda + Glue)

Ticket data needs heavier, schema-aware transformation than logs, so the Lambda trigger hands processing off to an AWS Glue PySpark Visual ETL job rather than transforming inline.

```
glue.start_job_run(  
    JobName="automate_etl_support_tickets",  
    Arguments={"--input_file_path": s3_input_path}  
)
```

- Column names are standardized to a consistent analytics schema.
- Records with invalid num_interactions values are filtered out.
- Priority-field typos are corrected using SQL CASE logic inside the Glue job.
- Output is coalesced into a single file and written as Parquet + Snappy.
- A Glue Data Quality ruleset (CarePlusSupportTicketsDQ) gates the output before it is treated as pipeline-complete.

9. Data Models & Schemas

This section documents the actual fields used in downstream analytics, based on the processed output consumed by Power BI. The log parser extracts 14 fields in total from each raw entry; the fields below are the core subset exposed through current analytics and dashboards.

9.1 Support Logs — Core Schema

Column	Data Type	Description	Example
timestamp	timestamp	Time of the log event	2025-07-01 10:23:45
log_level	string	Severity of the event (standardized)	INFO / DEBUG / WARNING
response_time	integer (ms)	System response time in milliseconds	983
cpu_utilization	decimal (%)	CPU utilization at time of event	55.12
user_agent	string	Originating client / integration	PostmanRuntime, curl, Python-urllib

Note: the remaining fields captured by the 14-field regex extraction (e.g. additional request/session metadata) are not yet surfaced in the current Power BI layer and are candidates for future dashboard expansion.

9.2 Support Tickets — Core Schema

Column	Data Type	Description	Example
ticket_id	string	Unique ticket identifier	TCK0701004
agent	string	Assigned support agent	Ananya
priority	string	Ticket priority (standardized)	High / Medium / Low
issue_category	string	Type of issue raised	Bug Report, Payment Failure
channel	string	Customer interaction channel	Email, Chat, Phone, Web Form
status	string	Current ticket status	Open / Resolved / Escalated
num_interactions	integer	Number of interactions on the ticket	4
resolution_time	integer (min)	Time taken to resolve the ticket	1917

9.3 Warehouse Model — Current State

The current Redshift Serverless implementation loads support_logs and support_tickets as independent analytical tables, incrementally updated by the Lambda loader described in Section 10. Cross-dataset analysis (e.g. logs correlated with ticket volume) is currently done at the Power BI / query level rather than via a pre-built dimensional model.

A dimensional model (fact_support_tickets, fact_support_logs, dim_date, dim_agent) would allow faster, pre-joined BI queries at larger scale and is a natural next step once query patterns stabilize.

10. Data Quality Framework

Dataset	Rule	Action	Severity	Business Impact
Logs	Invalid / unparseable timestamp	Reject	High	Incorrect time-based analysis
Logs	Negative response_time	Reject	High	Invalid performance KPI
Logs	Invalid CPU utilization value	Reject	Medium	Skewed infrastructure metrics
Logs	log_level typo (e.g. INFO, EROR)	Standardize	Medium	Inconsistent categorization
Logs	Duplicate record	Remove	High	Inflated log volume
Tickets	Invalid num_interactions	Reject	High	Incorrect workload analysis
Tickets	Priority-field typo	Standardize	Medium	Incorrect priority reporting
Tickets	Empty / malformed structure	Block	Critical	Prevents corrupted downstream data
Both	Schema validation on Parquet output	Gate	Critical	Blocks load if structure is invalid

10.1 Data Quality Flow

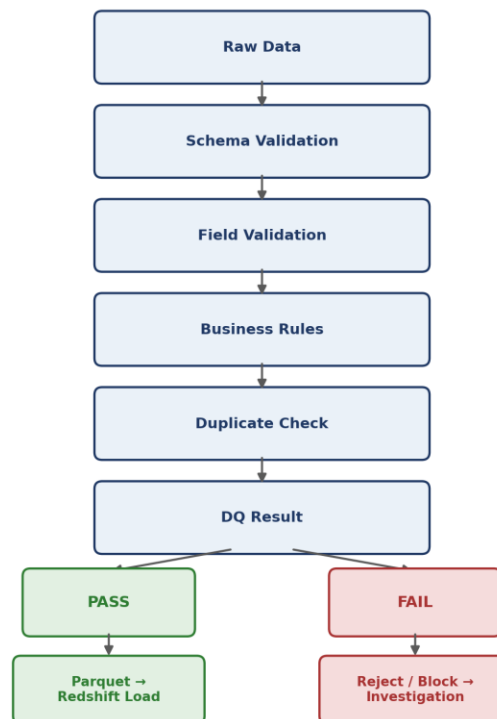


Figure 6: Data quality gate — pass records continue to Redshift; failed records are blocked and investigated.

11. Incremental Loading to Redshift

A dedicated incremental-load Lambda is triggered whenever new Parquet files land in the processed zone. It connects to Redshift Serverless using credentials sourced from environment variables (never hard-coded) and loads only the newly arrived records, so Redshift always reflects the latest processed state without full reloads.

```
conn = psycopg2.connect(  
    host=os.environ["REDSHIFT_HOST"],  
    port=5439,  
    database=os.environ["REDSHIFT_DATABASE"],  
    user=os.environ["REDSHIFT_USER"],  
    password=os.environ["REDSHIFT_PASSWORD"]  
)
```

Credentials shown above are illustrative placeholders only; no real host names, passwords, or ARNs are included anywhere in this report.

11.1 Current approach to idempotency

Date-tracked ingestion (Section 7.1) prevents re-uploading already-processed source dates, which is the primary safeguard against duplicate processing today.

PRODUCTION ENHANCEMENT — NOT YET IMPLEMENTED

```
S3 Event -> Event ID / Object Key -> Idempotency Store  
-> Already processed? --YES--> Skip  
                    --NO---> Process
```

A dedicated idempotency store (e.g. a DynamoDB table keyed on object key + ETag) would guard against duplicate processing if the same S3 event is delivered more than once — a known at-least-once delivery behavior of S3 event notifications.

12. Monitoring & Observability

- Amazon CloudWatch captures execution logs for every Lambda invocation across all four functions (logs ETL, tickets trigger, and both incremental loaders).
- Successful and failed runs are traceable end-to-end from S3 event to Redshift write, which was essential for diagnosing the psycopg2 issue described in Section 19.
- Glue job runs and Data Quality results are visible in the Glue console for each execution.

13. Operational Runbook

Documented recovery procedures for the three failure scenarios most relevant to this pipeline. Scenario 3 reflects an incident that actually occurred during the build (see Section 19).

13.1 Scenario 1 — Lambda ETL Failure

```
S3 file uploaded -> Lambda triggered -> Lambda FAILED
```

- Open CloudWatch Logs and identify the failed invocation.
- Check the exception / error message.
- Verify the input S3 object exists and is well-formed.
- Validate the file structure against the expected format.

- Fix the transformation issue and redeploy if needed.
- Reprocess the failed file and verify the processed Parquet output.
- Confirm the downstream Redshift load completed.

13.2 Scenario 2 — Glue Job Failure

```
S3 CSV -> Lambda -> Glue -> FAILED
```

- Open AWS Glue and locate the failed job run.
- Review Glue job logs for the transformation/schema error.
- Validate the source CSV against the expected schema.
- Correct the transformation logic and re-run the Glue job.
- Verify the Data Quality result and confirm Parquet output.

13.3 Scenario 3 — Redshift Load Failure (actual incident)

```
Redshift load failed -> Check Lambda CloudWatch logs -> Check error type
-> Check dependencies -> Check credentials/connectivity
-> Check Redshift availability -> Retry load -> Validate row count
```

This procedure is generalized directly from the real psycopg2 dependency failure documented in Section 19, where the missing-dependency branch of this runbook was the actual root cause.

14. Alerting Strategy

14.1 Current implementation

- CloudWatch execution logs for every Lambda and Glue run.
- Lambda and Glue failure visibility in their respective consoles.
- Manual troubleshooting through log review (as used in Section 19).

NO AUTOMATED ALERTING IMPLEMENTED YET

14.2 Production target

Component	Alert Condition	Action
Lambda	Invocation failure	SNS notification
Glue	Job failure	SNS notification
Data Quality	DQ rule failure	Stop downstream load
Redshift	Load failure	SNS notification
Pipeline	No data received in expected window	Monitoring alert

Lambda / Glue -> CloudWatch -> CloudWatch Alarm -> SNS -> Email / Operations Team

15. Business Glossary

Term	Definition	Source
Support Ticket	A customer-reported issue or request handled by support	Support Tickets
Resolution Time	Time required for an agent to resolve a ticket	Support Tickets
Response Time	Time taken by the system to respond to a request	Support Logs
Ticket Priority	Business severity assigned to a ticket	Support Tickets

Term	Definition	Source
Log Level	Severity/category of a system event (INFO, DEBUG, WARNING)	Support Logs
Escalated Ticket	A ticket requiring higher-level intervention	Support Tickets
Support Channel	Channel through which the customer contacted support	Support Tickets
CPU Utilization	Percentage of compute capacity in use at event time	Support Logs

16. Security

16.1 Implemented in this build

- Dedicated IAM user (dev-ingestion-user) with programmatic access only, scoped to ingestion.
- Separate scoped IAM roles per Lambda / Glue job rather than one shared role.
- S3 access restricted to the project bucket; no public bucket policies.
- Redshift credentials read from environment variables, never hard-coded in source.

16.2 Production-grade improvements

PRODUCTION ENHANCEMENT — NOT YET IMPLEMENTED

- AWS Secrets Manager for Redshift and other credentials instead of environment variables.
- Stricter least-privilege IAM policies scoped per resource ARN.
- Encryption at rest (S3/Redshift) and in transit enforced by policy, plus CloudTrail auditing and KMS-managed keys.

Credentials, ARNs, and account identifiers are intentionally redacted throughout this report.

17. Cost Management / FinOps

The project was built entirely within a fixed AWS free-credit allowance (expiry 09/08/2026), which made cost discipline an engineering requirement, not an afterthought.

17.1 Daily Shutdown Checklist

- Priority 1 (highest cost risk): Redshift Serverless, EC2 instances
- Priority 2: Glue jobs and crawlers left running
- Priority 3: S3, EFS, unattached Elastic IPs

17.2 Design trade-offs

Design Choice	Benefit	Trade-off
Lambda	No idle infrastructure	Runtime / execution-time limits
Glue	Handles heavier transformations	More expensive per run than Lambda
Parquet	Compression + analytics performance	Not human-readable
Redshift Serverless	Managed warehouse, no cluster ops	Can incur usage cost if left running
S3	Low-cost durable storage	Requires a catalog/query layer to be useful

17.3 Verified teardown after validation

Resource	Post-teardown count
Redshift Serverless	0
S3 buckets	0
Lambda functions	0
Lambda layers	0

Resource	Post-teardown count
Glue jobs	0
CloudWatch log groups	0
EC2 instances	0
Elastic IPs	0
NAT Gateways	0
RDS databases / snapshots	0
-\$0.00 AWS Cost Explorer, Aug 1–20, 2026	

18. Validation & Reconciliation

To confirm the pipeline was not just “running” but correctly propagating every record end-to-end, a controlled incremental-load test was run and reconciled at each stage.

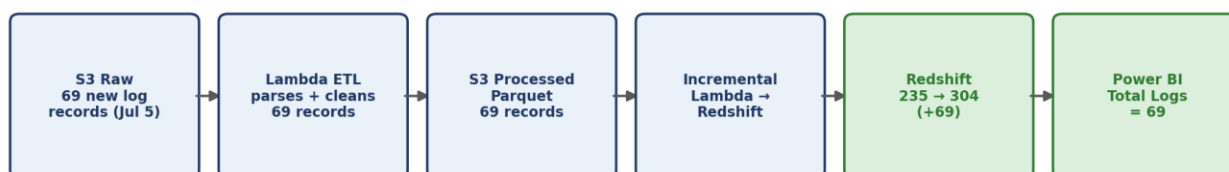


Figure 7: Record-level reconciliation for the July 5 incremental batch — the same 69 records are traceable from S3 through to the Power BI dashboard.

235 Redshift before	304 Redshift after	+69 New records loaded	69 Power BI Total Logs
-------------------------------	------------------------------	----------------------------------	----------------------------------

Conclusion: the controlled July 5 test batch reconciled successfully across S3, Lambda, Parquet, the incremental Redshift load, and the Power BI dashboard, with no observed record loss or duplication in the validated batch.

19. Testing & Test Cases

The following test matrix reflects rules actually implemented in the ETL logic (Sections 7, 8 and 10). Row-level rule tests are validated by design/code review; the Redshift load test was validated through the live reconciliation in Section 18.

Test	Input	Expected Result	Status
Valid log file	Well-formed .log entry	Processed successfully	PASS
Invalid timestamp	Malformed timestamp	Record rejected	PASS
Negative response time	response_time = -100 ms	Record rejected	PASS
Duplicate record	Duplicate row in batch	Duplicate removed	PASS
log_level typo	"INFO", "EROR"	Standardized to INFO / ERROR	PASS
Ticket priority typo	e.g. "HGIH"	Standardized to High	PASS
Invalid interactions	Negative num_interactions	Record rejected	PASS
Malformed ticket structure	Empty / broken CSV row	DQ failure, load blocked	PASS
Incremental Redshift load	69 new processed log records	Redshift +69 rows, Power BI = 69	PASS
Automated alerting on failure	Simulated Lambda failure	SNS notification sent	PLANNED

20. Troubleshooting Case Study: psycopg2 in Lambda

20.1 Incident

The incremental-load Lambda failed at runtime with `Runtime.ImportModuleError: No module named 'psycopg2'` when attempting to connect to Redshift.

20.2 Root cause

The Lambda execution environment does not include third-party libraries by default; psycopg2 (the PostgreSQL/Redshift driver) was not part of the base runtime and had no attached layer.

20.3 Resolution

- Built a psycopg2 Lambda Layer compatible with Python 3.12, x86_64 architecture.
- Attached the layer to the incremental-load Lambda function.
- Redeployed and re-tested the function against Redshift.
- Execution succeeded, and CloudWatch confirmed the Redshift load completed.

20.4 Result

“Data successfully loaded into Redshift” — confirmed via CloudWatch logs and the subsequent 235 → 304 row-count reconciliation in Section 18.

21. Power BI Business Analytics

21.1 Support Logs Dashboard

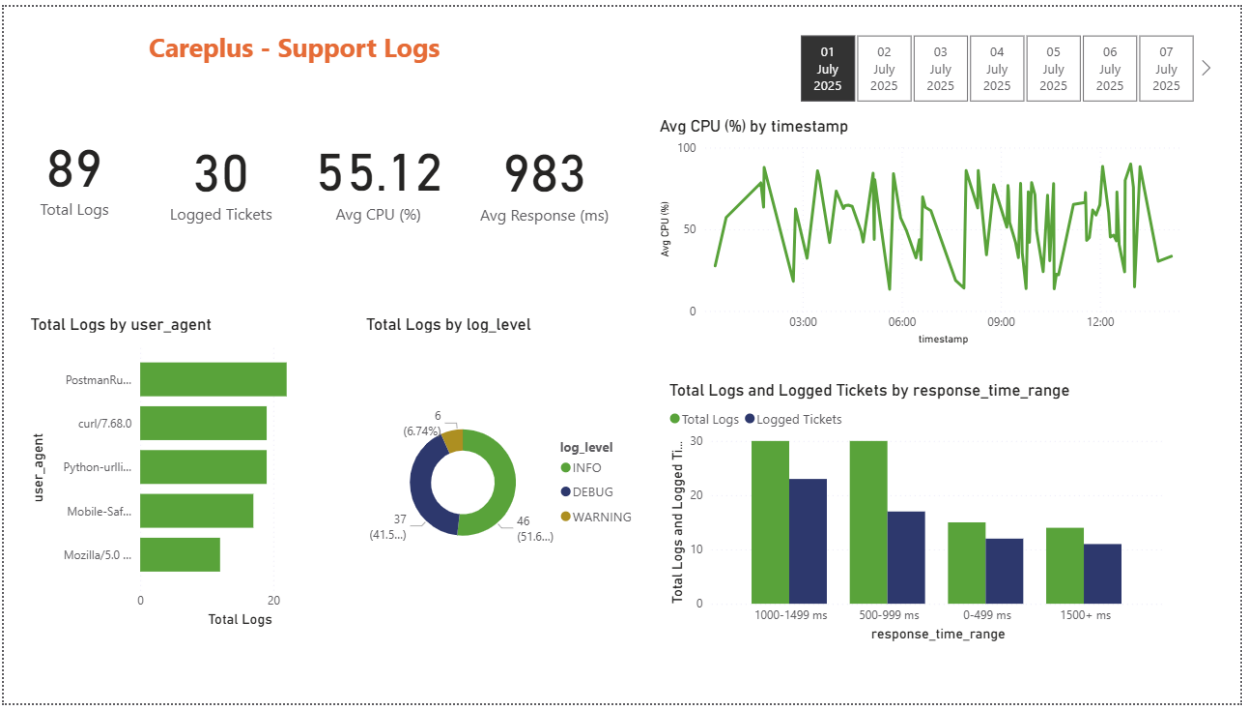


Figure 8: CarePlus Support Logs dashboard — daily-tab view for July 1, 2025.

89 Total Logs	30 Logged Tickets	55.12 Avg CPU %	983 Avg Response (ms)
------------------	----------------------	--------------------	--------------------------

The dashboard breaks logs down by user agent and log level, and cross-tabulates total logs and logged tickets against response-time bands — directly surfacing where response-time risk concentrates.

21.2 Ticket Insights Dashboard

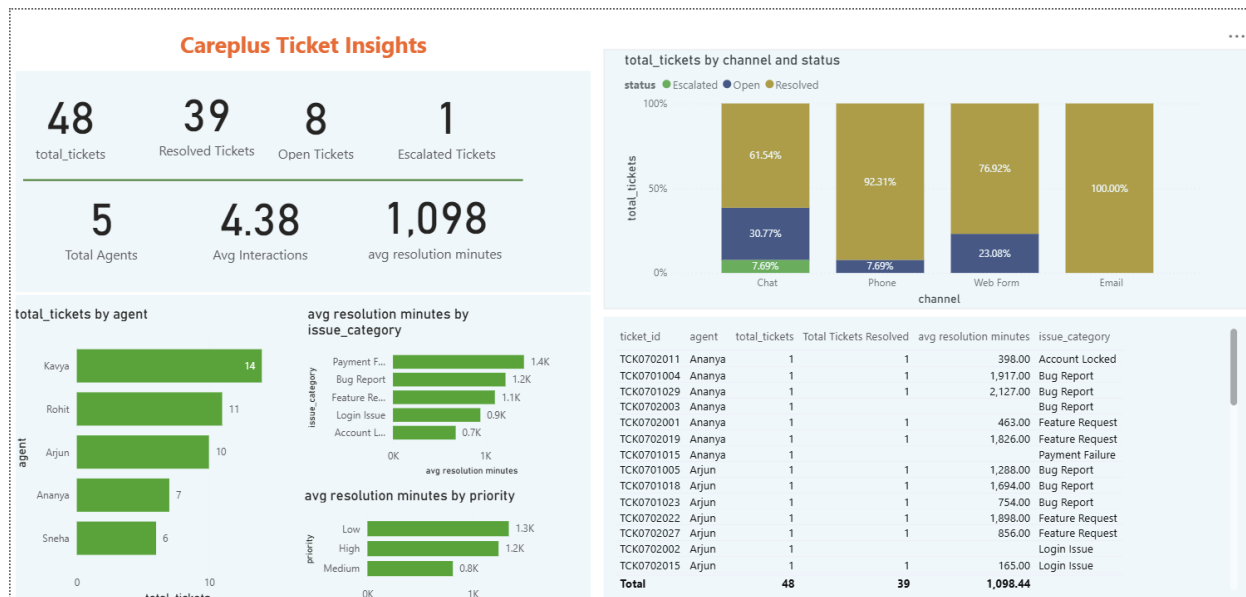


Figure 9: CarePlus Ticket Insights dashboard — agent workload, resolution time, and channel/status breakdown.

48 Total Tickets	39 Resolved	8 Open	1 Escalated
---------------------	----------------	-----------	----------------

- 5 agents handled 48 tickets at an average of 4.38 interactions and 1,098 minutes average resolution time.
- Email and Web Form channels show the highest resolution rates, while Chat has the highest escalation share — a direct operational signal for staffing/triage.

22. Business Use Cases Enabled

22.1 Operations

- Which support channels generate the most tickets, and which escalate most often?
- What proportion of tickets remain unresolved at any given time?
- Which priority categories have the longest average resolution time?

22.2 Technology / Engineering

- Which log levels dominate, and where are error events concentrated?
- Is CPU utilization correlated with response time?

- Which user agents or integrations generate the most support-triggering activity?

22.3 Management

- Are ticket volumes trending up or down over time?
- Are response times improving or worsening against informal SLAs?
- Which operational areas need proactive investigation before they become customer-facing issues?

23. AWS Resource Inventory — During Project Execution

Note: the resources listed in this section represent the infrastructure used during project execution. The environment was fully torn down after validation (see Section 17.3) to avoid ongoing AWS usage charges; none of these resources are currently running.

Resource	Name	Details
IAM User	dev-ingestion-user	Programmatic access only; used for ingestion notebooks
S3 Bucket	careplus-data-srl	Region: ap-southeast-1; raw + processed zones, both pipelines
Lambda	automate_support_logs_ETL	Python 3.13, AWSSDKPandas layer; triggered on support-logs/raw/*.log
Lambda	automate_support_tickets_etl	Triggered on support-tickets/raw/*.csv; invokes Glue job
Lambda	Incremental Redshift loader	Triggered on processed-zone write; psycopg2 layer attached
Glue Job	automate_etl_support_tickets	Visual ETL (PySpark) for ticket standardization
Glue DQ Ruleset	CarePlusSupportTicketsDQ	Validates processed tickets output before load
Glue Database	careplus_db	Glue Data Catalog database for crawled tables
Redshift Serverless	CarePlus warehouse	Analytical serving layer for Power BI
CloudWatch	Lambda / Glue log groups	Execution logging and troubleshooting evidence

24. Architecture Decisions & Trade-offs

Service	Why chosen
S3	Durable, low-cost object storage; natural raw/processed zone boundary
Lambda	Event-driven, lightweight transformation for the semi-structured logs path
Glue (PySpark)	Managed Spark for heavier, schema-aware ETL on structured ticket data
Parquet + Snappy	Columnar, compressed format optimized for analytical querying
Redshift Serverless	Warehouse-style analytical serving without managing clusters
Athena	Ad-hoc SQL directly on S3 Parquet, no infrastructure to provision
CloudWatch	Native execution monitoring for every Lambda/Glue run
IAM	Fine-grained access control per function and role
Power BI	Business-facing consumption layer for operations and management

Why not EC2?

EC2 was intentionally avoided because both workloads are event-driven and intermittent rather than continuous. Lambda and Glue provide managed serverless execution, avoiding the need to maintain continuously running EC2 infrastructure for these intermittent workloads — which directly supports the Section 17 cost-management goals.

25. Challenges & Root Causes

Challenge	Root cause	Fix
Two data types, two shapes	Logs are semi-structured free text; tickets are structured CSV with heavier cleaning needs	Built two purpose-fit paths instead of forcing one pattern onto both
Re-ingesting old data	No tracking of what had already been uploaded	Date-tracker file pattern for deterministic, incremental ingestion
psycopg2 missing in Lambda	Library not included in base Lambda runtime	Custom Lambda Layer (Python 3.12, x86_64)
Cost risk during learning	Serverless resources can still accrue cost if left running (Redshift Serverless, Glue)	Daily shutdown checklist + full verified teardown

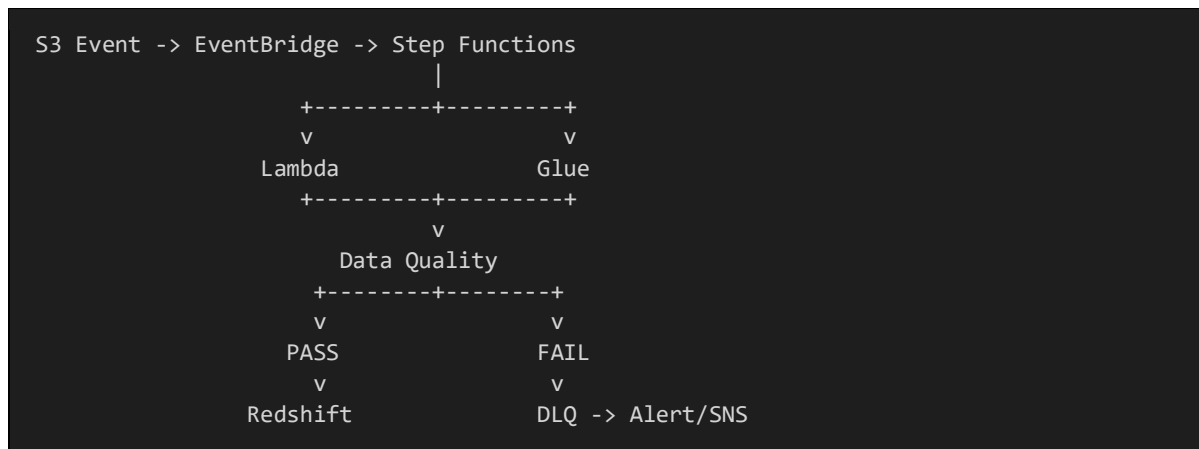
26. Limitations & Production Roadmap

26.1 Current limitations

- Incremental-loader source code lives in the Lambda console rather than a version-controlled GitHub repo.
- No automated CI/CD for deploying Lambda/Glue changes.
- No dead-letter queue for failed event processing.
- No event idempotency store to guard against duplicate S3 event delivery (Section 11.1).
- Limited automated alerting beyond CloudWatch logs (Section 14).
- S3 folder structure is source-first rather than zone-first, which limits partition efficiency at scale.

26.2 Production-Grade Target Architecture — V2

PRODUCTION ENHANCEMENT — NOT CURRENT ARCHITECTURE



27. Production Readiness Scorecard

Capability	Current	Production Target
Event-driven ingestion	Implemented	Implemented
Automated ETL	Implemented	Implemented
Data Quality	Implemented	Enhanced (more rules, per-record logging)
Incremental loading	Implemented	Enhanced (idempotent, replay-safe)
Monitoring	Implemented (CloudWatch)	Enhanced (dashboards + metrics)
Alerting	Partial (manual log review)	Automated via SNS
Idempotency	Partial (date-tracker only)	Dedicated idempotency store
Failure handling	Partial (manual reprocessing)	Dead-letter queue + retries
Secrets management	Partial (environment variables)	AWS Secrets Manager

Capability	Current	Production Target
CI/CD	Not implemented	GitHub Actions
Orchestration	Not implemented	Step Functions
Data lake zoning	Partial (source-first layout)	Bronze / Silver / Gold zones

28. How I Explain This Project in 90 Seconds

"I built an event-driven AWS pipeline for CarePlus insurance support data. Daily logs and ticket extracts land in S3, where an event trigger kicks off processing — logs go through a Python/Lambda ETL, while tickets are handed off to a Glue PySpark job for heavier structured cleaning. Both outputs are written as validated Parquet, gated by an AWS Glue Data Quality ruleset. A separate incremental Lambda then loads only the new processed records into Redshift Serverless, which Power BI reads from. I validated the whole chain end-to-end: a batch of 69 new log records moved the Redshift count from 235 to 304, and Power BI reconciled the same 69 records on the dashboard. Along the way I diagnosed and fixed a missing psycopg2 dependency by building a custom Lambda Layer. Everything was built serverless-first to control cost, and I fully tore the environment down after validation — confirmed at \$0.00 in Cost Explorer. I've also documented the gaps — alerting, idempotency, CI/CD — as a clear production roadmap."

29. Conclusion

CarePlus demonstrates a complete, validated data engineering lifecycle — from raw insurance support data to a governed, analytics-ready warehouse and a business-facing Power BI layer — built and torn down entirely within a fixed cost budget. Beyond the pipeline itself, the project is documented the way production data platforms are: schemas, a business glossary, a data quality framework, an operational runbook, and a production-readiness scorecard that honestly separates what is implemented from what is planned. That combination of working automation, real troubleshooting, and honest documentation directly supports a transition into a full-time Data Analyst / Data Engineer role.

30. Author & Contact

Name: Sahajahanur Rahman Laskar (Sahajan)

Background: 5 years' experience in insurance operations (MIS Executive / Customer Service, HDFC ERGO General Insurance, 2020–2025)

Email: connectingsrl@gmail.com

LinkedIn: linkedin.com/in/sahajahanur-laskar

GitHub: github.com/Sahajahanur